

C5-DEC CAD end-user manual

Accessible guide for daily workflows

This report consolidates and reorganizes C5-DEC end-user documentation from docs/manual into a single workflow-oriented manual.

General information

Name of the company	itrust Abstractions Lab (ALab)
Project title	C5-DEC CAD
Type	Report (REP)
Reference	3D3M
Version	1.0
State	Final
Owner	ALab
Authors	C5-DEC team
Application date	20/03/2026
Classification	TLP:CLEAR

Document history

Version	Date	Author	Modifications
1.0	23/03/2026	C5-DEC team	Summary of first release: • Created consolidated end-user manual structure from docs/manual content • Added workflow-oriented navigation and quick command reference • Integrated manual screenshots into a self-contained DocEngine report project

Approval

Name	Role	Responsibility	Signature
Arash	CTO	Product owner	—
Arash	CTO	Technical writer	—
Itzel	RD Scientist	Technical writer	—

Table of contents

Preface	8
Start here	8
Scope	8
1 Reader guide	9
1.1 Who this manual is for	9
1.2 How to navigate this document	9
1.3 Source coverage	9
1.4 Interfaces at a glance	10
2 Installation and workspace setup	11
2.1 Prerequisites for all paths	11
2.2 Procedure A: Docker + runner scripts	11
2.2.1 Steps	11
2.2.2 Common execution modes	12
2.2.3 Expected outcome	12
2.2.4 Checkpoints (if something fails)	12
2.3 Procedure B: VS Code dev container (recommended)	12
2.3.1 Required tools	12
2.3.2 Steps	12
2.3.3 Optional Poetry recovery	13
2.3.4 Expected outcome	13
2.3.5 Checkpoints	13
2.4 Procedure C: choose a devcontainer variant	13
2.4.1 Variant 1: C5-DEC CAD dev container (default)	13
2.4.2 Variant 2: C5-DEC CAD cryptography dev container (PQC)	13
2.4.3 Variant 3: DocEngine-focused container	14
2.5 Procedure D: workspace setup after installation	14
2.5.1 Checkpoints	14
2.6 Procedure E: legacy pipx installation (deprecated)	15
2.6.1 Global steps	15

2.6.2	Platform notes	15
2.6.3	Legacy usage quick checks	16
2.7	Installation verification checklist	16
3	Quick start and interfaces	17
3.1	Launch modes	17
3.2	Start commands	17
3.3	TUI usage pattern	17
3.4	GUI usage pattern	18
3.5	Third-party GUI integration	19
4	Module guides	20
4.1	Common Criteria Toolbox (CCT)	20
4.1.1	Procedure A: browse CC content via GUI	20
4.1.2	Procedure B: create checklist via TUI	23
4.1.3	Procedure C: run checklist and export flow via CLI	25
4.2	Secure software development life cycle (SSDLC)	26
4.2.1	Procedure A: bootstrap a new C5-DEC SSDLC project	26
4.2.2	Procedure B: maintain artifact lifecycle	27
4.2.3	Procedure C: publish technical specs and reports	27
4.3	DocEngine	29
4.3.1	Procedure A: create and configure a template	29
4.3.2	Procedure B: write content and render	31
4.3.3	Procedure C: ETR generation pipeline	32
4.4	Project resource management (PM)	35
4.4.1	Procedure A: convert OpenProject time report	35
4.4.2	Procedure B: consolidate and filter reports	36
4.4.3	Procedure C: compute cost report and analyze	37
4.5	Cyber-physical system security assessment (CPSSA)	38
4.5.1	Procedure A: create a threat model template	39
4.5.2	Procedure B: generate report and DFD	39
4.5.3	Procedure C: FAIR calibration and quantitative risk analysis	39
4.6	Cryptography	40
4.6.1	Procedure A: integrity and signature workflow	40
4.6.2	Procedure B: encryption and decryption workflow	40
4.6.3	Procedure C: secret sharing and NaCl signing	41
4.7	Information security management system (ISMS)	41
4.7.1	Procedure A: verify ISMS folder against reference list	41

4.7.2	Procedure B: generate activity report CSV	42
4.7.3	Procedure C: convert Word tags to hyperlinks	42
4.8	Software bill of materials (SBOM)	43
4.8.1	Procedure A: generate and import SBOM	43
4.8.2	Procedure B: diff and validate SBOM	44
4.9	CRA compliance	44
4.9.1	Procedure A: create and complete CRA checklist	44
4.9.2	Procedure B: integrate SBOM and export compliance report	45
4.9.3	Procedure C: build CRA technical documentation	45
5	Troubleshooting	47
5.1	Fast triage order	47
5.2	Container build failures	47
5.3	Poetry environment issues	47
5.4	Doorstop validation errors	47
5.5	GUI port conflicts	48
5.6	General recovery guidance	48
6	Command reference	49
6.1	Global launcher	49
6.2	CCT	49
6.3	SSDLC and publishing	49
6.4	PM	49
6.5	CPSSA	50
6.6	Cryptography	50
6.7	CRA and SBOM	50
6.8	Notes	50
7	References	51
7.1	Source manual pages	51
7.2	Figure source	51

List of Figures

2.1	Devcontainer selector	13
2.2	PQC devcontainer option	14
2.3	CLI help output	15
3.1	TUI main menu	18
3.2	GUI browser module	19
4.1	CCT GUI browser	21
4.2	CCT GUI selection	22
4.3	CCT GUI preview	23
4.4	CCT submenu	24
4.5	Checklist creation	24
4.6	Evaluation checklist	25
4.7	SSDLC menu	28
4.8	Spec browser	28
4.9	Traceability home	29
4.10	DocEngine report template open in VS Code	31
4.11	DocEngine compiled report example	32
4.12	DocEngine ETR output	34
4.13	OpenProject assistant	35
4.14	Conversion view	36
4.15	PM menu	37
4.16	Consolidation assistant	37
4.17	Cost report in Excel	38
4.18	Verification sheet	38
4.19	DocList assistant	42
4.20	Activity report	43
4.21	Traceability statistics	45
4.22	Traceability graph	46

List of Tables

Preface

This manual provides an end-user oriented path through C5-DEC workflows.

The content is reorganized from docs/manual to improve discoverability while keeping the same functional scope.

Start here

1. Use [Reader guide](#) for navigation strategy.
2. Use [Installation and workspace setup](#) when preparing or repairing an environment.
3. Use [Quick start and interfaces](#) for launch paths.
4. Use [Module guides](#) for feature workflows.
5. Use [Troubleshooting](#) to resolve common failures.
6. Use [Command reference](#) for copy/paste command lookup.

Scope

This report covers user-facing operation of C5-DEC modules and publishing workflows documented in the manual set.

It does not change product behavior or introduce new normative requirements.

Chapter 1

Reader guide

This manual is a consolidated, end-user oriented guide built from the content in the C5-DEC manual pages.

1.1 Who this manual is for

This version is optimized for users who already know the platform and need quick access to module workflows, commands, and troubleshooting paths.

1.2 How to navigate this document

Use this flow for fast task completion:

1. Read *Installation and workspace setup* in Chapter 2 only when you need to set up or repair an environment.
2. Use *quick start and interfaces* in Chapter 3 for CLI, TUI, and GUI launch paths.
3. Jump to *Module guides* in Chapter 4 for feature-specific workflows.
4. Use *Troubleshooting* in Chapter 5 when commands or environments fail.
5. Keep *Command reference* in Chapter 6 open for copy/paste operations.

1.3 Source coverage

The chapters in this report are reorganized from the following source pages:

- [docs/manual/installation.md](#)
- [docs/manual/start.md](#)
- [docs/manual/cct.md](#)
- [docs/manual/ssdlc.md](#)
- [docs/manual/pm.md](#)
- [docs/manual/cpssa.md](#)
- [docs/manual/cryptography.md](#)
- [docs/manual/isms.md](#)
- [docs/manual/sbom.md](#)
- [docs/manual/cra.md](#)
- [docs/manual/troubleshooting.md](#)

1.4 Interfaces at a glance

C5-DEC provides three primary interfaces:

- CLI for scriptable and automatable workflows.
- TUI for keyboard-driven interactive operation in a terminal.
- GUI for browser-based exploration and checklist workflows.

The manual content also documents third-party GUI usage patterns through Visual Studio Code, Zettlr, and Firefox integration paths.

Chapter 2

Installation and workspace setup

This chapter provides full installation procedures for all supported deployment models.

Use this decision order:

1. Docker + C5-DEC runner scripts (fastest for CLI/TUI/GUI use).
2. VS Code dev container workflow (recommended for advanced SSDLC, DocEngine, and integrated development).
3. Legacy pipx wheel installation (deprecated, use only when containerized workflows are not viable).

2.1 Prerequisites for all paths

- Python 3 installed.
- Git installed.
- Local copy of C5-DEC repository or release package.

2.2 Procedure A: Docker + runner scripts

This is the quickest path to launch C5-DEC in an isolated environment without manual dependency management.

2.2.1 Steps

1. Install Docker engine or Docker Desktop and ensure it is running.
2. Open a terminal in the project directory.
3. Ensure scripts are executable:

```
chmod +x build-c5dec.sh
chmod +x c5dec.sh
```

4. Build images:

```
./build-c5dec.sh
```

5. Launch C5-DEC:

```
./c5dec.sh  
./c5dec.sh help
```

2.2.2 Common execution modes

```
./c5dec.sh          # default help/entrypoint  
./c5dec.sh session  # interactive container session  
./c5dec.sh pqc      # post-quantum crypto entrypoint
```

2.2.3 Expected outcome

- C5-DEC starts in containerized mode with CLI/TUI/GUI entrypoints available.

2.2.4 Checkpoints (if something fails)

- Docker daemon unreachable: see [Troubleshooting](#) and the section on container build failures.
- Build fails on package/network step: see [Troubleshooting](#).

2.3 Procedure B: VS Code dev container (recommended)

Use this path for full development workflows and integrated tooling (Poetry, Quarto, SpecEngine, notebook workflows).

2.3.1 Required tools

1. Docker Desktop (or Docker engine).
2. Visual Studio Code.
3. Dev Containers extension for VS Code.

2.3.2 Steps

1. Clone repository:

```
git clone https://github.com/AbstractionsLab/c5dec.git
```

2. Start Docker Desktop/engine.
3. Open the repository folder in VS Code.
4. Run Dev Containers: Reopen in Container.
5. Select C5-DEC CAD dev container.
6. Wait for build/start completion.
7. Open terminal in container and validate:

```
c5dec -h  
c5dec -t  
c5dec -g
```

2.3.3 Optional Poetry recovery

If dependencies are missing in the container shell:

```
poetry install
poetry shell
```

2.3.4 Expected outcome

- Local filesystem is mounted in container.
- c5dec commands run directly in integrated terminal.
- Full C5-DEC feature set available.

2.3.5 Checkpoints

- No Reopen in Container prompt: see [Troubleshooting](#) section on Dev Container detection.
- c5dec: command not found: see [Troubleshooting](#) section on Poetry environment activation.

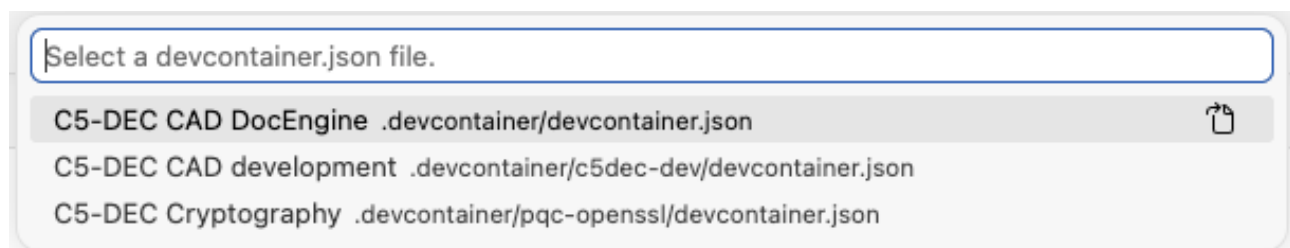


Figure 2.1: Devcontainer selector

2.4 Procedure C: choose a devcontainer variant

2.4.1 Variant 1: C5-DEC CAD dev container (default)

Use for: - full module coverage, - SSDLC and Doorstop operations, - most cryptography, transformer, and PM workflows.

2.4.2 Variant 2: C5-DEC CAD cryptography dev container (PQC)

Use for post-quantum OpenSSL/OQS operations.

Steps: 1. Reopen in container. 2. Select C5-DEC CAD cryptography dev container. 3. Validate available PQC algorithms:

```
openssl list -signature-algorithms -provider oqsprovider
openssl list -kem-algorithms -provider oqsprovider
```

Expected outcome: - OQS-enabled OpenSSL commands available in container shell.

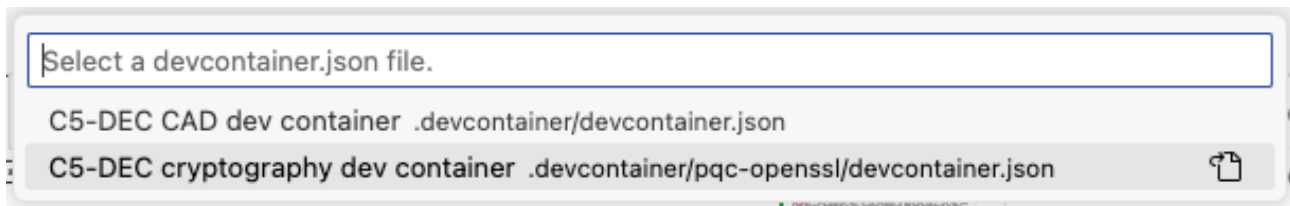


Figure 2.2: PQC devcontainer option

2.4.3 Variant 3: DocEngine-focused container

A dedicated `docEngine.Dockerfile` is available for document-generation-focused workflows.

Use for: - Quarto rendering, - SpecEngine publish pipeline, - DocEngine-centric CI/CD or lightweight publishing environments.

Build/run:

```
docker build -f docEngine.Dockerfile -t c5dec-docengine .
docker run --rm -it -v "$(pwd)":/workspace c5dec-docengine bash
```

Expected outcome: - Quarto and publication toolchain ready for rendering and spec publication tasks.

2.5 Procedure D: workspace setup after installation

Run these once per workspace (regardless of deployment path).

1. Unpack and rename project folder as needed.
2. Ensure git repository exists:

```
git init .
```

3. Ensure Poetry environment is active in container sessions when needed:

```
poetry shell
```

4. Run quick launch checks:

```
c5dec -h
c5dec -t
c5dec -g
```

2.5.1 Checkpoints

- Import warnings in editor: select Poetry interpreter (see [Troubleshooting](#)).
- GUI not reachable or port conflict: see [Troubleshooting](#) GUI sections.

```
usage: c5dec [-h] [-j PATH] [-V] [-v] [-t] [-g] <command> ...

C5-DEC CAD for Computer-Assisted Design and Development.

positional arguments:
  <command>
    new                ssdlc - create new C5-DEC project with containerization, scripts, templates and configuration
    docengine           docengine - create new DocEngine report, presentation, or CRA technical documentation template
    timerep             pm - convert OpenProject xls time report to C5-DEC time sheet
    consolidate         pm - consolidate all C5-DEC time sheets from c5dec/input directory
    costrep            pm - compute cost report from C5-DEC time sheet
    view               cct - retrieve Common Criteria (CC) item (class/family/component/element) by ID or name
    validate           cct - run CC component choice validation routine
    checklist          cct - create evaluation checklist in YAML+Markdown, editable via TUI
    export             cct - export work unit (WU) evaluation checklist to spreadsheet: no selection -> all WUs exported.
    etr               cct - process WU evaluation checklist spreadsheet to generate ETR parts for C5-DEC DocEngine
    cra               cra - create and export CRA essential requirements checklist for EU Cyber Resilience Act compliance
    sbom              cra - generate, import, compare, and manage Software Bill of Materials (SBOM) for CRA compliance
    transform         transformer - import, export, convert and compile using integrated doorstop, pandoc and quarto
    cpsa              cpsa - threat modelling and risk analysis on top of OSS: OWASP pytm, threatgile and pyfair
    crypto            cryptography - run crypto operations or launch C5-DEC interactive session to use GnuPG, Kryptor, Cryptomator and OQS-OpenSSL for PQC

options:
  -h, --help            show this help message and exit
  -j PATH, --project PATH set path to project root (default: /home/alab/c5dec)
  -V, --version         show program's version number and exit
  -v, --verbose         enable verbose logging (default: 0)
  -t, --tui            run textual user interface (TUI) (default: True)
  -g, --gui            run graphical user interface (GUI) (default: False)
```

Figure 2.3: CLI help output

2.6 Procedure E: legacy pipx installation (deprecated)

Use only if Docker/containers are unavailable.

2.6.1 Global steps

1. Install pipx:

```
python3 -m pip install --user pipx
python3 -m pipx ensurepath
```

2. Optional Doorstop installation:

```
pipx install doorstop==3.0b10
```

3. Install C5-DEC wheel from unpacked release directory:

```
pipx install ./c5dec-<version>-py3-none-any.whl
```

2.6.2 Platform notes

2.6.2.1 GNU/Linux

- Python 3.8+ typically available.
- Install with same pipx commands above.
- Launch:

```
c5dec
```

2.6.2.2 macOS

- Ensure Python 3.8+ available.
- Install and launch exactly as GNU/Linux.

2.6.2.3 Windows

1. Install Python via Microsoft Store.
2. Install pipx with:

```
python3 -m pip install --user pipx  
python3 -m pipx ensurepath
```

3. Install wheel:

```
pipx install .\c5dec-<version>-py3-none-any.whl
```

4. Launch with c5dec in terminal.

2.6.3 Legacy usage quick checks

```
c5dec -h  
c5dec -t  
c5dec -g
```

2.7 Installation verification checklist

Use this as final pass/fail validation:

1. c5dec -h returns help without command-not-found errors.
2. TUI opens using c5dec -t.
3. GUI starts and can be reached on expected port.
4. For dev container workflows, Poetry environment is active and dependencies resolve.
5. For DocEngine workflows, Quarto render commands execute successfully in selected environment.

Chapter 3

Quick start and interfaces

This chapter consolidates launch and interface usage from the quick start guide.

3.1 Launch modes

C5-DEC supports three primary launch modes:

- CLI for direct commands and automation.
- TUI for menu-driven operation in terminal.
- GUI for browser-based interaction.

3.2 Start commands

```
c5dec -h  
c5dec -t  
c5dec -g
```

3.3 TUI usage pattern

The TUI is optimized for keyboard navigation and module-first workflows. Use it when you want to operate without browser context and keep actions in one terminal session.

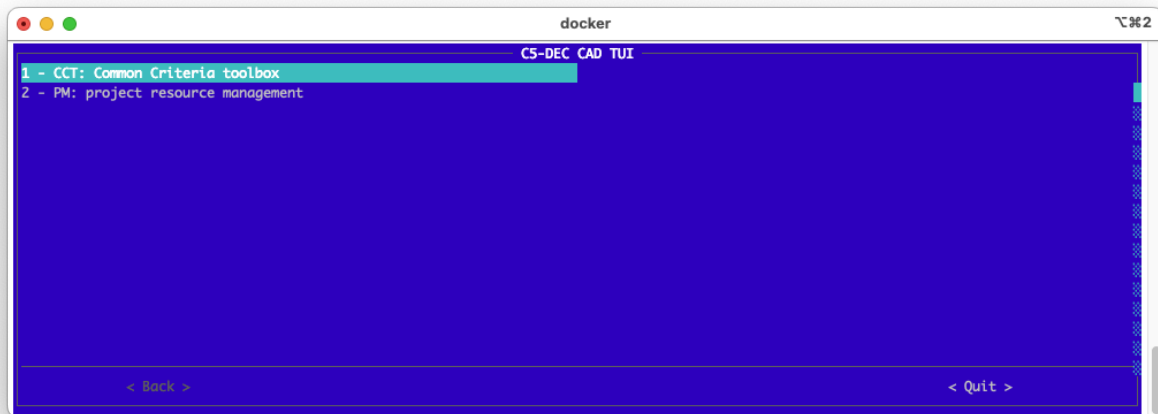


Figure 3.1: TUI main menu

3.4 GUI usage pattern

The GUI provides browser pages for CC browsing and evaluation lab operations. It is useful for interactive selection and formatted preview of selected CC content.

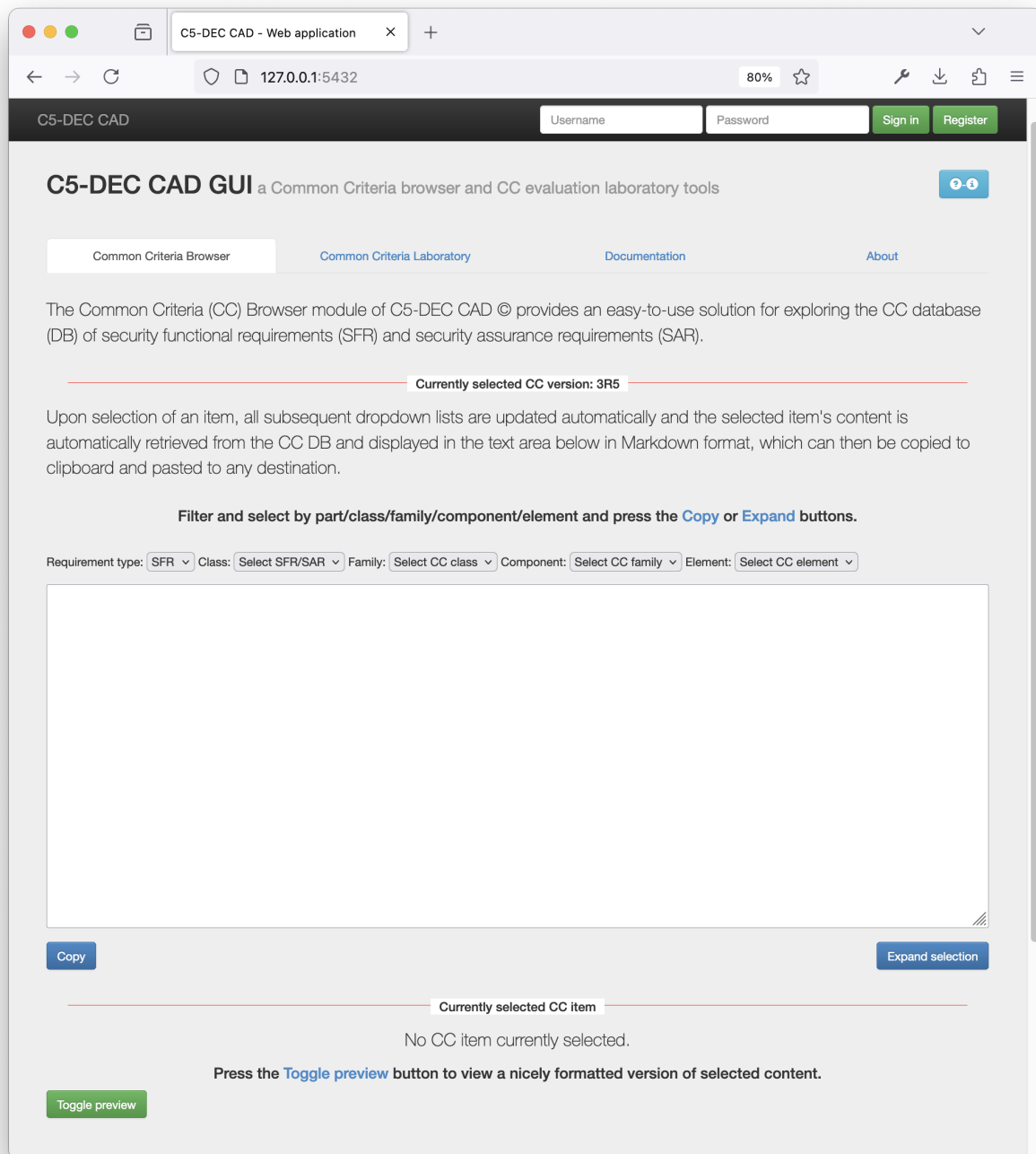


Figure 3.2: GUI browser module

3.5 Third-party GUI integration

The manual includes practical usage with:

- Visual Studio Code,
- Zettlr,
- Firefox.

These integrations are useful for editing, previewing, and publishing report-oriented artifacts.

Chapter 4

Module guides

This chapter expands each module into actionable procedures.

All steps are reorganized from the source manual pages and keep the same operational scope.

4.1 Common Criteria Toolbox (CCT)

Use CCT to browse Common Criteria content, validate component selections, create evaluation checklists, and generate ETR inputs.

Primary use:

- browse CC classes, families, components, and elements,
- create evaluation checklists,
- export checklist data,
- generate ETR input parts.

Key commands:

```
c5dec view <ID>
c5dec validate -d <IDs>
c5dec checklist -c <prefix> --id <IDs> --info <info>
c5dec export myChecklist 3R5 -p ALC_CMC.1
c5dec etr -f CMC -t DocStruct Acronyms Glossary
```

4.1.1 Procedure A: browse CC content via GUI

1. Launch the GUI:

```
c5dec -g
```

2. Open 127.0.0.1:5432 (or localhost:5432) in a browser.
3. In the CCT browser, select filters in this order:

- Requirement type (SFR or SAR)
- Class
- Family
- Component

- Element

4. Use **Copy** to copy the current markdown content, or **Expand selection** to include descendants.
5. Use **Toggle preview** to inspect formatted content.

Expected outcome: - CC text appears in markdown, updates automatically on each filter change, and can be copied/exported to downstream documentation.

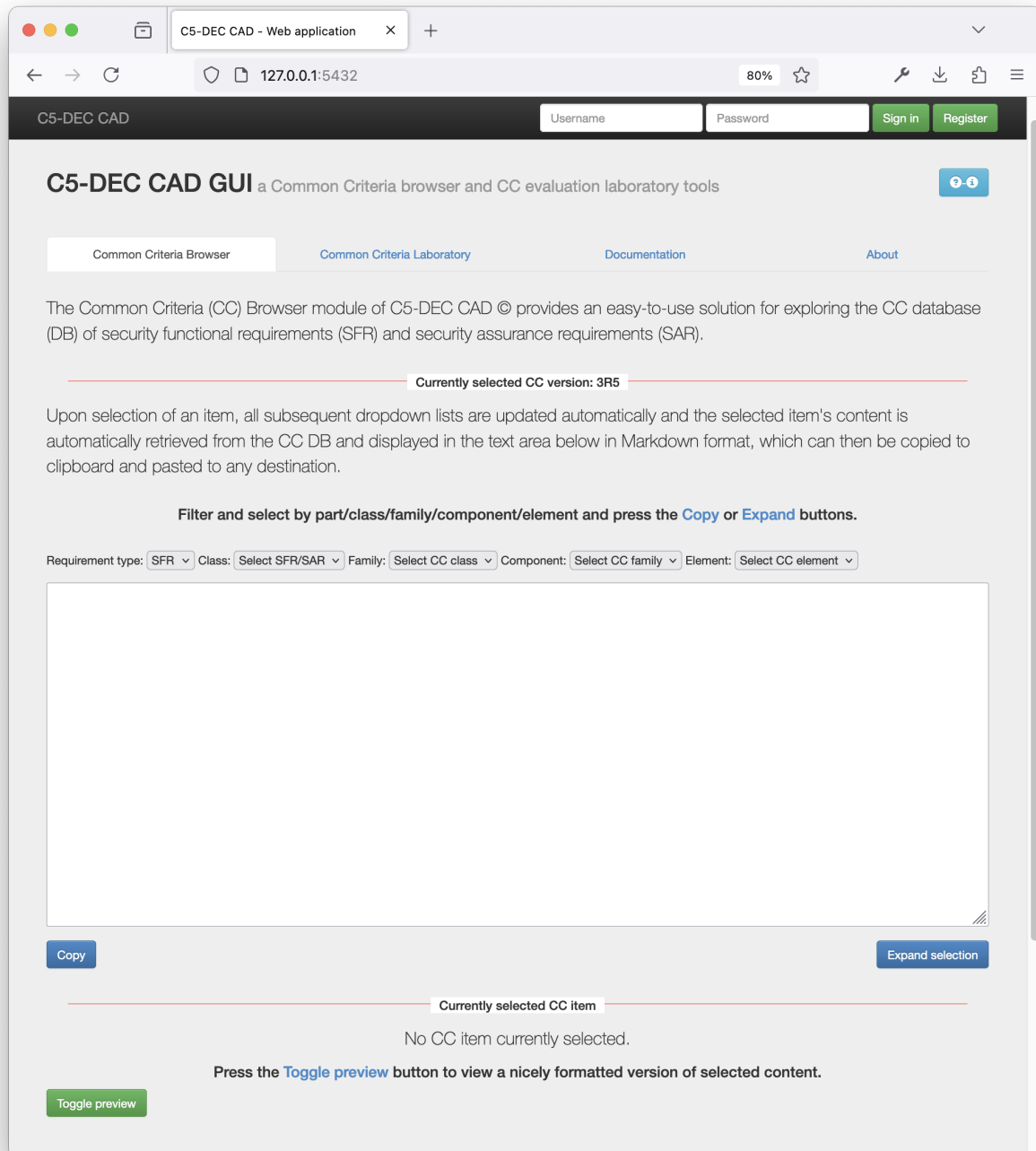


Figure 4.1: CCT GUI browser

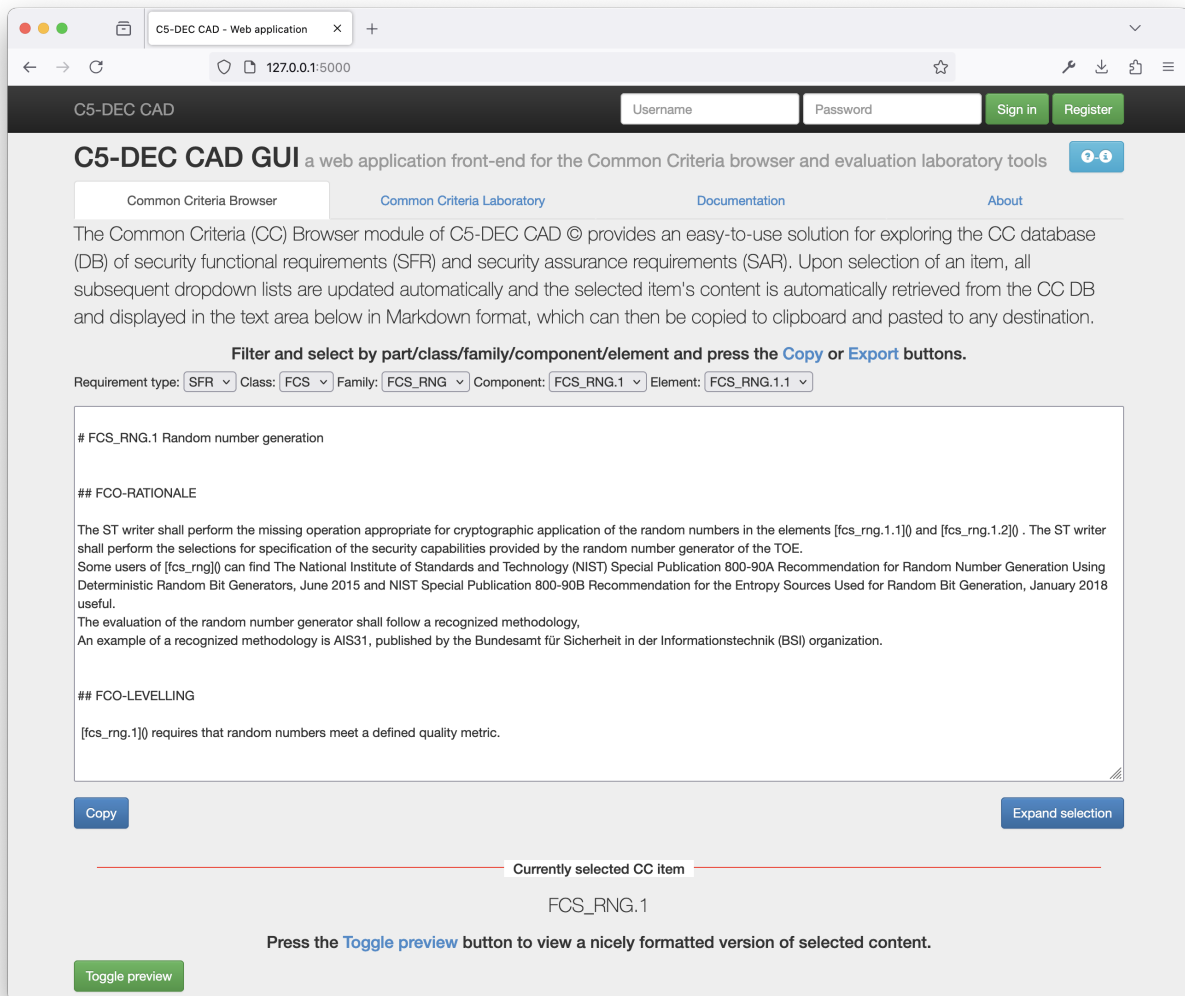


Figure 4.2: CCT GUI selection

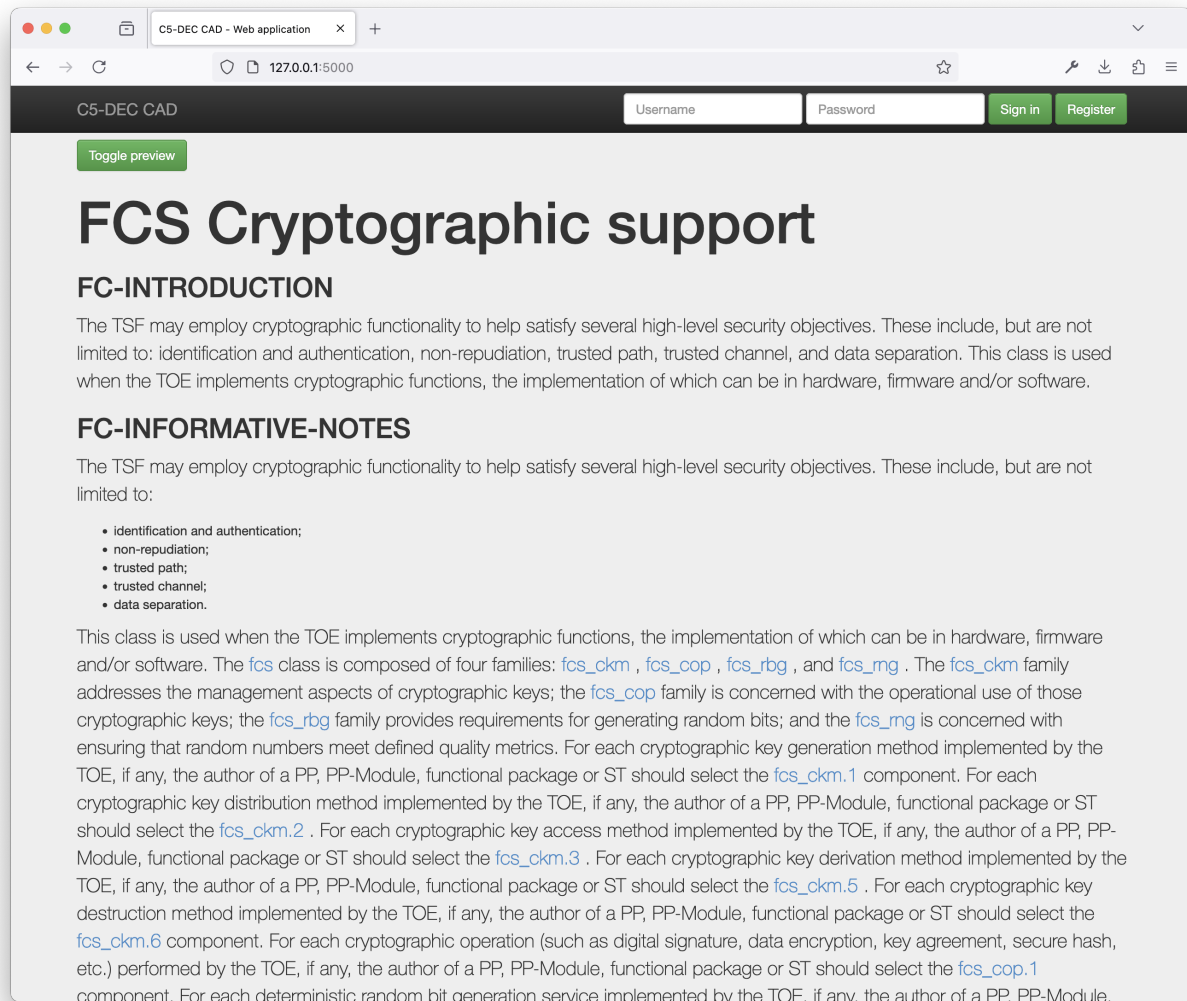


Figure 4.3: CCT GUI preview

4.1.2 Procedure B: create checklist via TUI

1. Launch TUI and navigate to Common Criteria Toolbox.
2. Open Create Evaluation Checklist.
3. Select either:
 - an assurance package under Packages, or
 - a custom component set under Select Item/s.
4. Confirm status bar shows a valid selection.
5. Provide checklist identity when prompted:
 - Evaluation Identifier (<eid>)
 - Creator
 - Creation Date
6. Proceed to Evaluation Checklist to review work units and enter evidence/verdicts (pass, fail, inconclusive).

Expected outcome: - A Doorstop checklist document is created with one item per selected evaluation work unit.

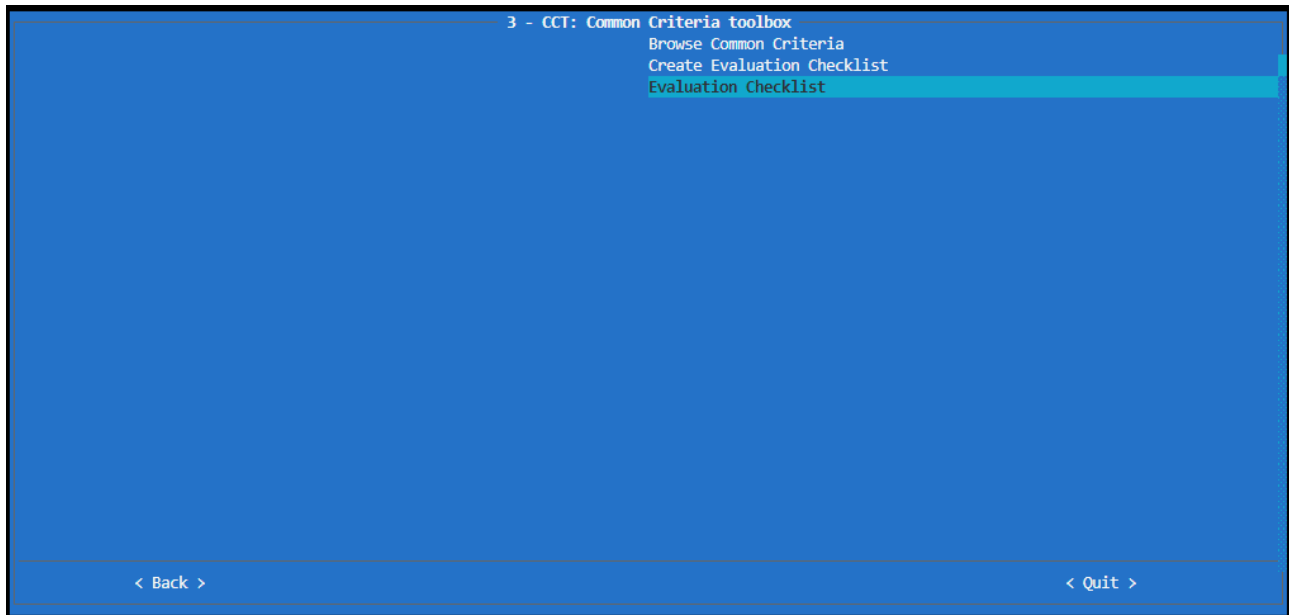


Figure 4.4: CCT submenu

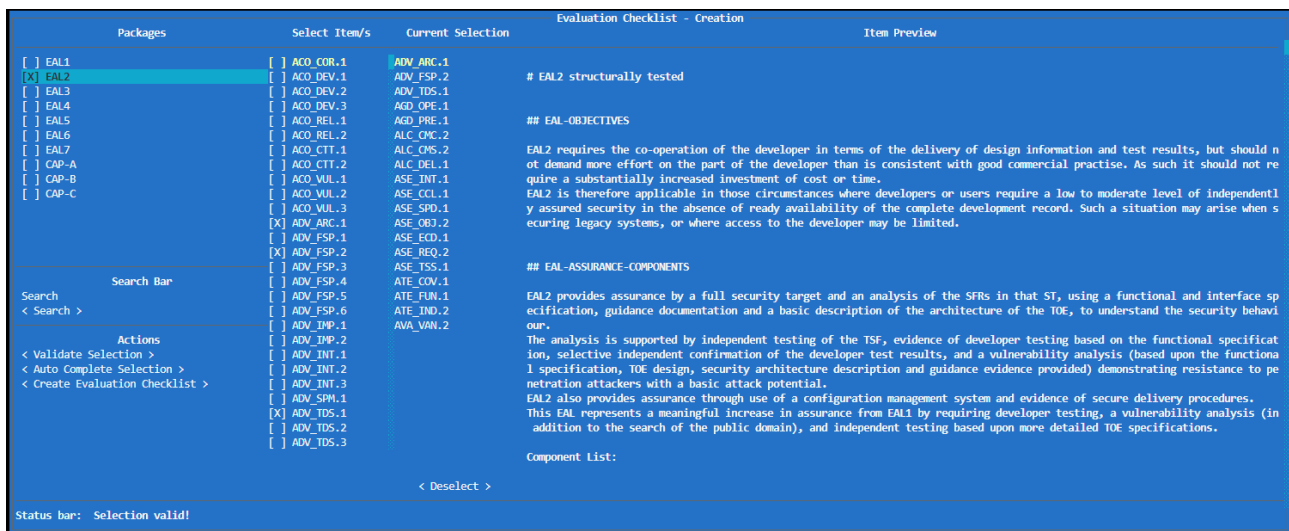


Figure 4.5: Checklist creation

Checklist/s	Evaluation Items	Evaluation Evidence	Evaluation Checklist	Work Unit
ARCANE	ADV_ARC.1-1	## Security Architecture Description		EVALUATOR ACTION ELEMENT
BRITTE	ADV_ARC.1-2			
DAVINCI	ADV_ARC.1-3	### Detailed Overview		
MAGNET	ADV_ARC.1-4	- **Document:** "BRITE_Security_Architecture_Overview_v1.0.pdf"	ADV_ARC.1.1E	
NEBULA	ADV_ARC.1-5	- Description: This document provides a detailed overview of BRITE's security architecture, illustrating its alignment with the Security Functional Requirements (SFRs).	# ADV_ARC.1.1E	
NEXAR	ADV_FSP.2-1			The evaluator shall confirm that the information provided meets all requirements for content and presentation of evidence.
QUICKMAP	ADV_FSP.2-2			
TRACER	ADV_FSP.2-3			
ZENLITH	ADV_FSP.2-4	### Abstraction Levels		
	ADV_FSP.2-5	- **Evidence:** "BRITE_Detail_Mapping_Matrix_v1.2.xlsx"		
	ADV_FSP.2-6	- Description: A matrix detailing how the security architecture's level of detail corresponds to the SFR-enforcing abstractions in the functional specification and TOE design.		CONTENT OR DEVELOPER ELEMENT
	ADV_FSP.2-7			
	ADV_FSP.2-8			
	ADV_FSP.2-9	## Functional Specification Alignment	ADV_ARC.1.1C	
	ADV_FSP.2-10		# ADV_ARC.1.1C	
	ADV_TDS.1-1	### Self-Protection Functionality		
	ADV_TDS.1-2	- **Document:** "BRITE_Self-Protection_Framework_v2.1.docx"		
	ADV_TDS.1-3	- Description: A document describing the self-protection mechanisms within BRITE, especially those affecting the TSFI, including executable image and file security measures.		
	ADV_TDS.1-4			
	ADV_TDS.1-5			
	ADV_TDS.1-6			
	ADV_TDS.1-7	### TSFI Description		
	ADV_TDS.1-8	- **Documentation:** "BRITE_TSFI_Comprehensive_Guide_v1.3.docx"		
	AGD_OPE.1-1	- Description: Comprehensive documentation of functionalities accessible through the TSFI, with detailed explanations.		WORK UNIT TASK
	AGD_OPE.1-2			
	AGD_OPE.1-3			
	AGD_OPE.1-4	< Pass >		
	AGD_OPE.1-5	< Fail >		
		< Inconclusive >		
< Load Evaluation Checklist/s >				The evaluator shall examine the security architecture description to determine that the information provided in the evidence is presented at a level of detail commensurate with the descriptions of the SFR-enforcing abstractions contained in the functional specification.
Status bar: Project BRITTE successfully loaded and validated!			Evaluation Progress	0%

Figure 4.6: Evaluation checklist

4.1.3 Procedure C: run checklist and export flow via CLI

1. Inspect CLI help:

```
c5dec -h
```

2. Browse and validate CC choices:

```
c5dec view <ID>
c5dec validate -d <IDs>
```

3. Create a checklist with metadata:

```
c5dec checklist -c <prefix> --id <IDs> --info project=MyTOE date=2026-03-01
```

4. Maintain checklist state:

```
c5dec checklist <prefix> --list
c5dec checklist <prefix> --edit <ID>
c5dec checklist <prefix> --update
c5dec checklist <prefix> --status
c5dec checklist <prefix> --validate
c5dec checklist <prefix> --publish <path>
```

5. Export spreadsheet from selected components/classes:

```
c5dec export myChecklist 3R5 -p ALC_CMC.1
c5dec export myChecklist 3R5 -c ALC_AVA
```

6. Generate ETR markdown parts from checklist spreadsheet:

```
c5dec etr -f CMC -t DocStruct Acronyms Glossary
```

Expected outcome: - Checklist artifacts and spreadsheet exports are created. - ETR markdown fragments are generated for DocEngine inclusion.

4.2 Secure software development life cycle (SSDLC)

Use SSDLC to create project scaffolding, manage Doorstop artifacts, maintain links, and publish traceable technical specifications.

Primary use:

- manage repositories and artifact items,
- define and maintain item relations,
- publish technical specifications and traceability outputs,
- use SpecEngine utilities for browser, statistics, and graph generation.

Key commands:

```
c5dec new -p <project_name> -u <user_directory>  
poetry run doorstop  
cd docs/specs && ./publish.sh
```

4.2.1 Procedure A: bootstrap a new C5-DEC SSDLC project

1. Create project bundle:

```
./c5dec.sh new
```

Or with explicit project/user:

```
./c5dec.sh new -p "newproject" -u "username"
```

2. Initialize git repository in the resulting project folder:

```
git init .
```

3. Open project in VS Code dev container and activate Poetry environment.

Expected outcome: - Project contains .devcontainer, docs/specs, docs/manual, docs/traceability, tests, and pre-configured DocEngine assets ready for use with the DocEngine module.

4.2.2 Procedure B: maintain artifact lifecycle

1. Use SSDLC menus (TUI) or Doorstop commands to:
 - create/update artifact items,
 - manage links,
 - review status and suspect links.
2. Keep artifact naming/prefixes aligned (MRS, SRS, ARC, SWD, TCS, TRP).

Expected outcome: - Traceability graph remains consistent from requirements through test evidence.

4.2.3 Procedure C: publish technical specs and reports

1. Go to specs folder:

```
cd /home/<username>/<projectname>/docs/specs
```

2. Run enhanced publication pipeline:

```
./publish.sh
```

3. Optionally run SpecEngine utilities directly:

```
python docs/specs/SpecEngine/c5publish.py  
python docs/specs/SpecEngine/c5browser.py  
python docs/specs/SpecEngine/c5traceability.py --html  
python docs/specs/SpecEngine/c5graph.py  
python docs/specs/SpecEngine/c5mermaid.py  
python docs/specs/SpecEngine/c5fingerprint.py
```

Expected outcome: - Published HTML specs, browser tables, traceability stats, graphs, and updated keyword rendering.

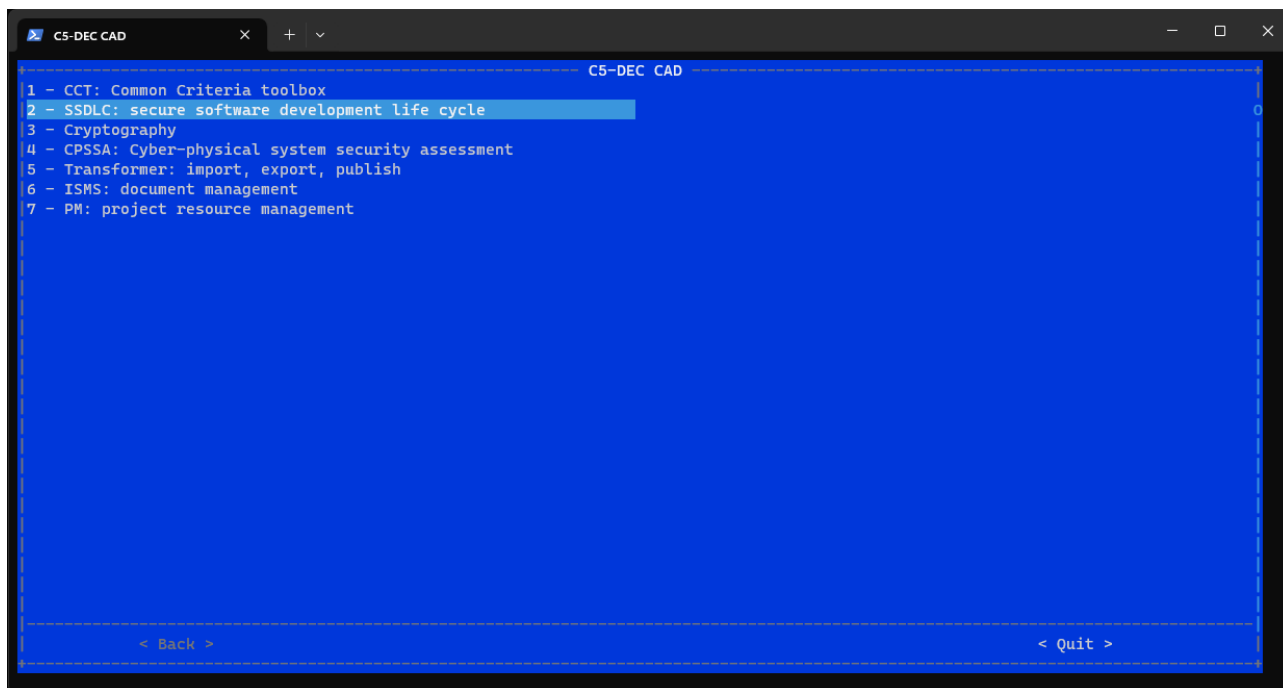


Figure 4.7: SSDLC menu

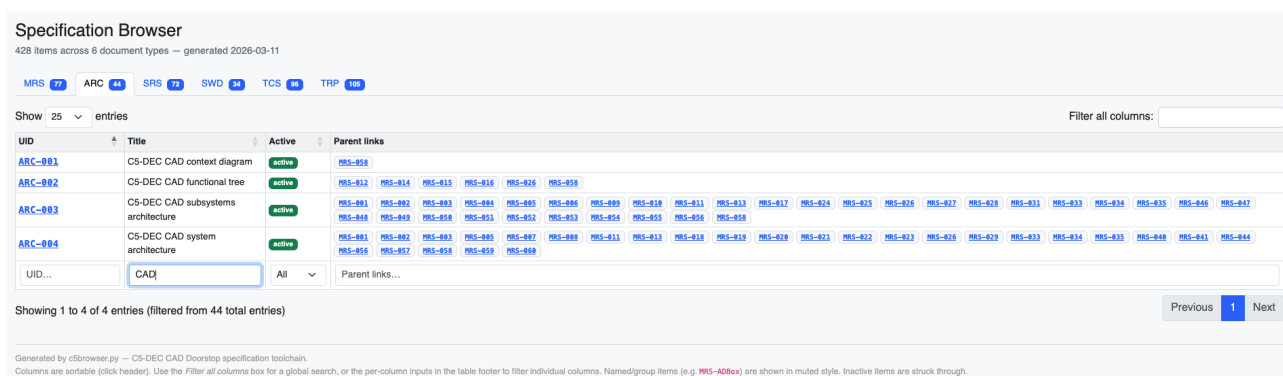
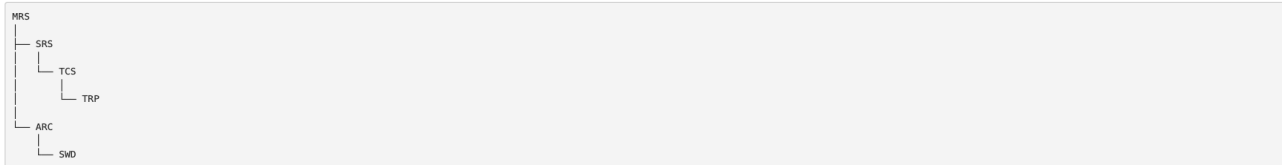


Figure 4.8: Spec browser

C5-DEC CAD traceability tooling reports:

- Specification browser
- Traceability statistics
- Traceability graph

Tree Structure:



Published Documents:

- ARC
- MRS
- SRS
- SWD
- TCS
- TRP

Item Traceability:

MRS	SRS	TCS	TRP	ARC	SWD
MRS-001 Diagram generation tool	SRS-049 Generate PlantUML Data Flow Diagram from architecture Items	TCS-090 TC: Test PlantUML DFD generation from ARC Items	TRP-093 TCER: PlantUML DFD generation from ARC Items		
MRS-001 Diagram generation tool	SRS-053 Generate interactive HTML graph for Doorstop specifications navigation				
MRS-001 Diagram generation tool				ARC-003 C5-DEC CAD	SWD-001 CCT class diagram

Figure 4.9: Traceability home

4.3 DocEngine

DocEngine is C5-DEC CAD's Quarto-based document publishing module. It is an independent component that scaffolds production-ready templates for generating technical documents — PDF reports, slide decks, and CRA Annex VII compliance documentation — from Markdown source files, with integrated Python pre/post-render scripts and extensive LaTeX customizations.

DocEngine requires the **C5-DEC DocEngine dev container**, which ships Quarto, TeX Live, Kryptor, and Cryptomator CLI on top of the base CAD container.

Primary use:

- scaffold report, presentation, and CRA technical documentation templates,
- configure document metadata (cover page, headers, footers, changelog),
- write content in Markdown chapters or slides,
- render to PDF, DOCX, and HTML from a single source,
- generate ETR documents through the CCT integration pipeline.

Key commands:

```

c5dec docengine report -n <name>
c5dec docengine presentation -n <name>
c5dec docengine cra-tech-doc -n <name>
c5dec docengine <type> -n <name> --standalone
c5dec docengine <type> -n <name> -d /path/to/destination
  
```

4.3.1 Procedure A: create and configure a template

1. Activate the Poetry environment inside the DocEngine dev container:

poetry shell

2. Scaffold the desired template type:

```
# Technical report
c5dec docengine report -n my-report

# Presentation
c5dec docengine presentation -n my-presentation

# CRA Annex VII technical documentation
c5dec docengine cra-tech-doc -n my-cra-doc
```

The template is created under `./docengine/<name>/` and a ZIP archive is placed alongside it. To change the destination directory:

```
c5dec docengine report -n my-report -d /path/to/output
```

3. Open `c5dec_config_v2.yml` at the root of the template folder and set document metadata:

```
document:
  title: "My Technical Report"
  subtitle: "Engineering study"
  version: "1.0"
  date: "2026-03-23"

changelog:
  - version: "1.0"
    date: "2026-03-23"
    changes:
      - "Initial release"
```

Use `c5dec_config.yml` (v1) for plain-string changelog entries. Use `c5dec_config_v2.yml` (v2, recommended) for structured changelog entries and automatic LaTeX escaping of special characters.

4. For a self-contained portable template that includes the full DocEngine dev container setup:

```
c5dec docengine report -n my-report --standalone
```

The recipient can open the resulting folder in VS Code and select “Reopen in Container” to get a fully configured environment.

Expected outcome: - Template folder created at `./docengine/<name>/` with pre-configured `_quarto.yml`, config files, chapter stubs, pre/post-render scripts, and LaTeX customization layer.

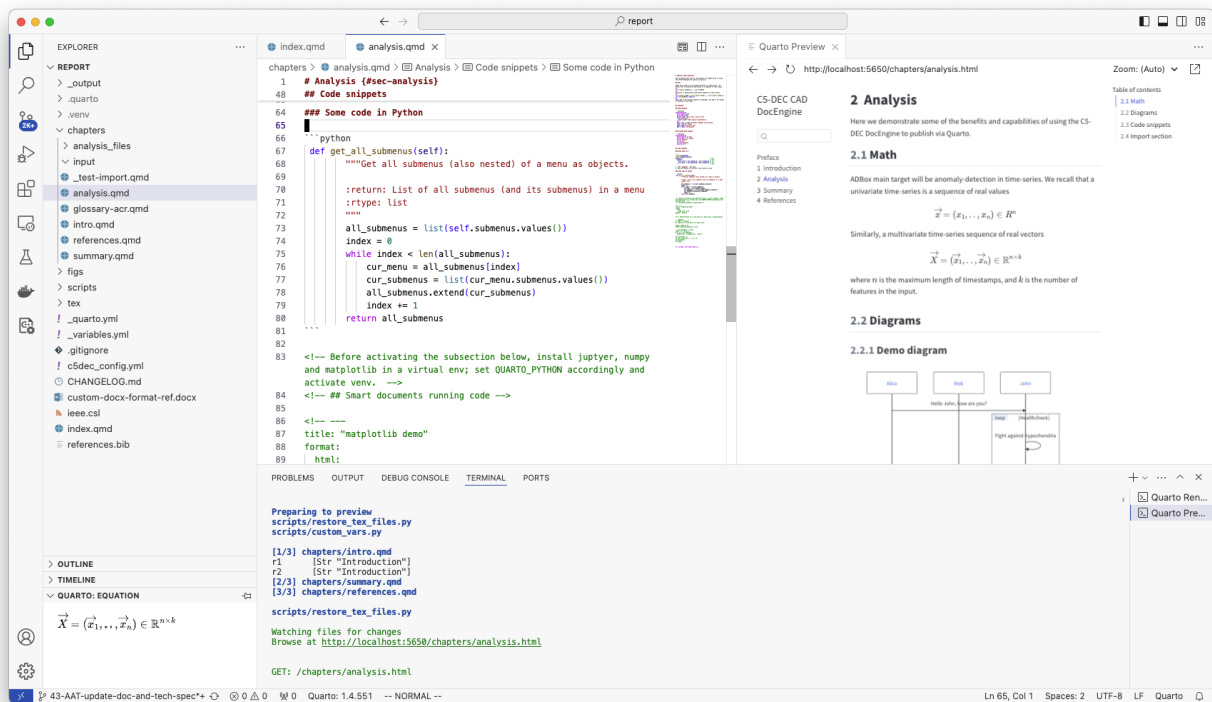


Figure 4.10: DocEngine report template open in VS Code

4.3.2 Procedure B: write content and render

1. Add report chapters as .qmd files inside the chapters/ folder. For presentations, add slide sections to the slides/ folder.
2. Register new chapters in _quarto.yml under book.chapters:

```
book:
  chapters:
    - index.qmd
    - chapters/intro.qmd
    - chapters/my-chapter.qmd
    - chapters/summary.qmd
    - chapters/references.qmd
```

3. Use _variables.yml for reusable values referenced in .qmd content with ?meta: ...:

```
project:
  name: "CyFORT"
  version: "1.3"
document:
  reference: "TR-007-2026"
```

In a chapter:

```
This is ?meta:project.name version ?meta:project.version.
Document reference: **?meta:document.reference**
```

4. Render to the desired format:

```
# PDF (via LuaLaTeX - full cover page, headers, footers)
quarto render ./docengine/<name>/index.qmd --to pdf

# HTML (Bootstrap Cosmo theme by default)
quarto render ./docengine/<name>/index.qmd --to html

# DOCX (uses custom-docx-format-ref.docx for styles)
quarto render ./docengine/<name>/index.qmd --to docx
```

Output is placed in ./docengine/<name>/_output/.

5. If Quarto picks up a different Python than the Poetry environment, set:

```
export QUARTO_PYTHON=$(which python)
```

Alternatively, use the Quarto VS Code extension (Command Palette → Quarto: Render Document) while inside the dev container.

Expected outcome: - PDF, HTML, and/or DOCX documents generated in _output/, with cover page, running headers/footers, and changelog from c5dec_config_v2.yml automatically applied.

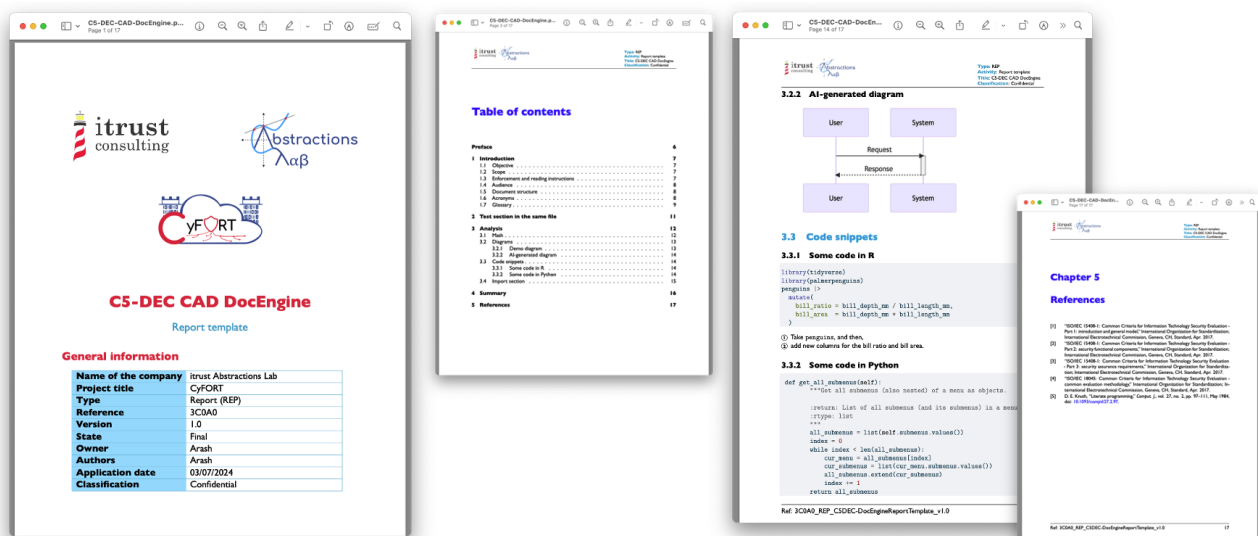


Figure 4.11: DocEngine compiled report example

4.3.3 Procedure C: ETR generation pipeline

DocEngine integrates with the Common Criteria Toolbox (CCT) to produce Evaluation Technical Reports (ETR).

1. Generate structured Markdown ETR parts from a completed evaluation checklist spreadsheet using the CCT etr subcommand:


```
c5dec etr -f CMC -t DocStruct Acronyms Glossary
```

2. Copy or move the generated Markdown parts into the `chapters/` folder of a DocEngine report template.
3. Register the parts in `_quarto.yml` under `book.chapters`.
4. Render to the final ETR format:

```
quarto render ./docengine/<name>/index.qmd --to pdf
```

Expected outcome: - A fully formatted ETR document compiled from CCT-generated Markdown parts via LaTeX.

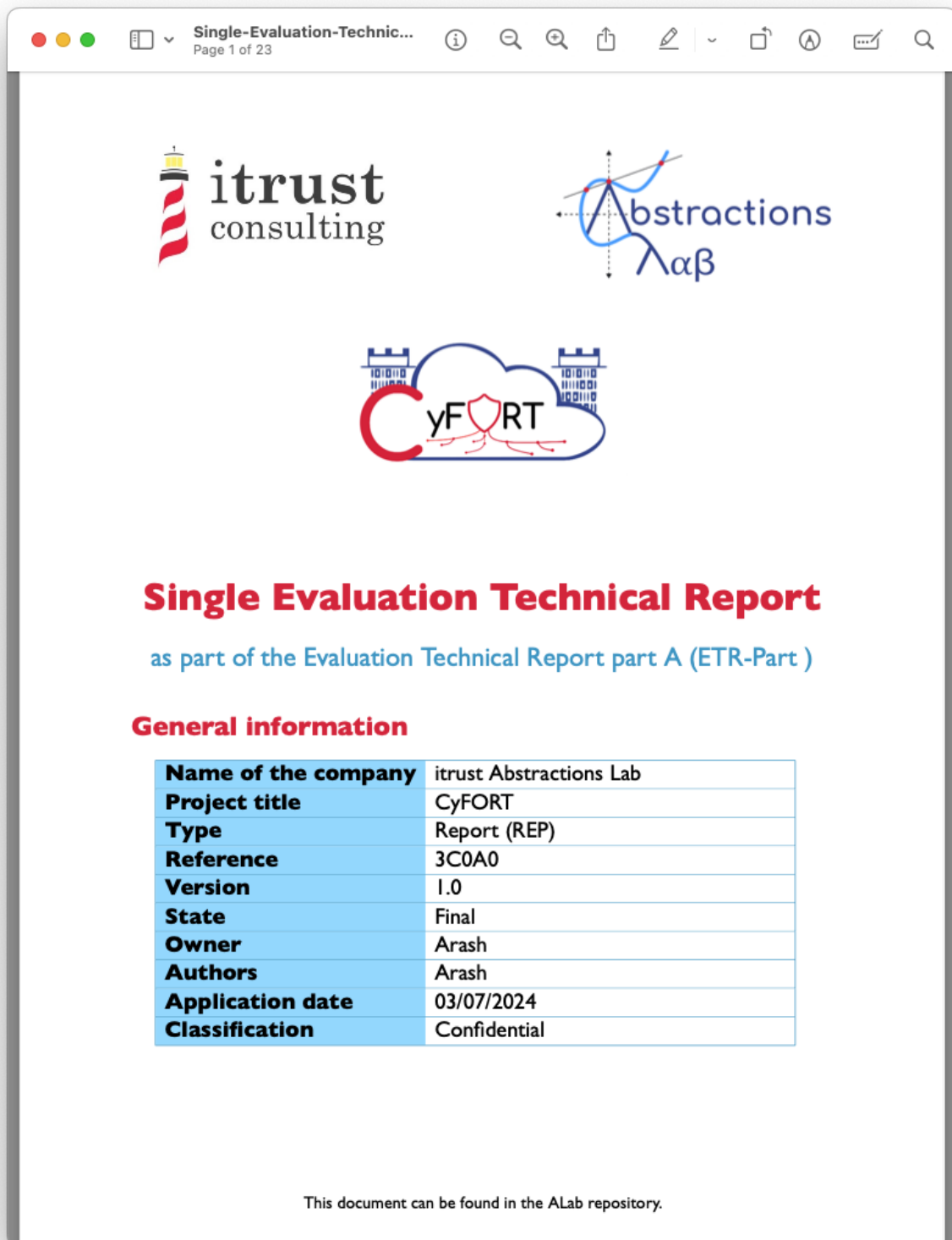


Figure 4.12: DocEngine ETR output

4.4 Project resource management (PM)

Use PM to convert OpenProject exports, consolidate timesheets, and compute cost reports.

Primary use:

- convert OpenProject time reports,
- consolidate time reports,
- compute cost reports.

Key commands:

```

c5dec timerep <openproject-export.xlsx>
c5dec consolidate <folder>
c5dec costrep <timesheet.xlsx>
  
```

4.4.1 Procedure A: convert OpenProject time report

1. Open PM module and select OpenProject time report assistant.
2. Provide absolute path to exported OpenProject file.
3. Trigger conversion.

CLI equivalent:

```
c5dec timerep <name>
```

Expected outcome: - Converted timesheet is written to current working directory using configured schema.

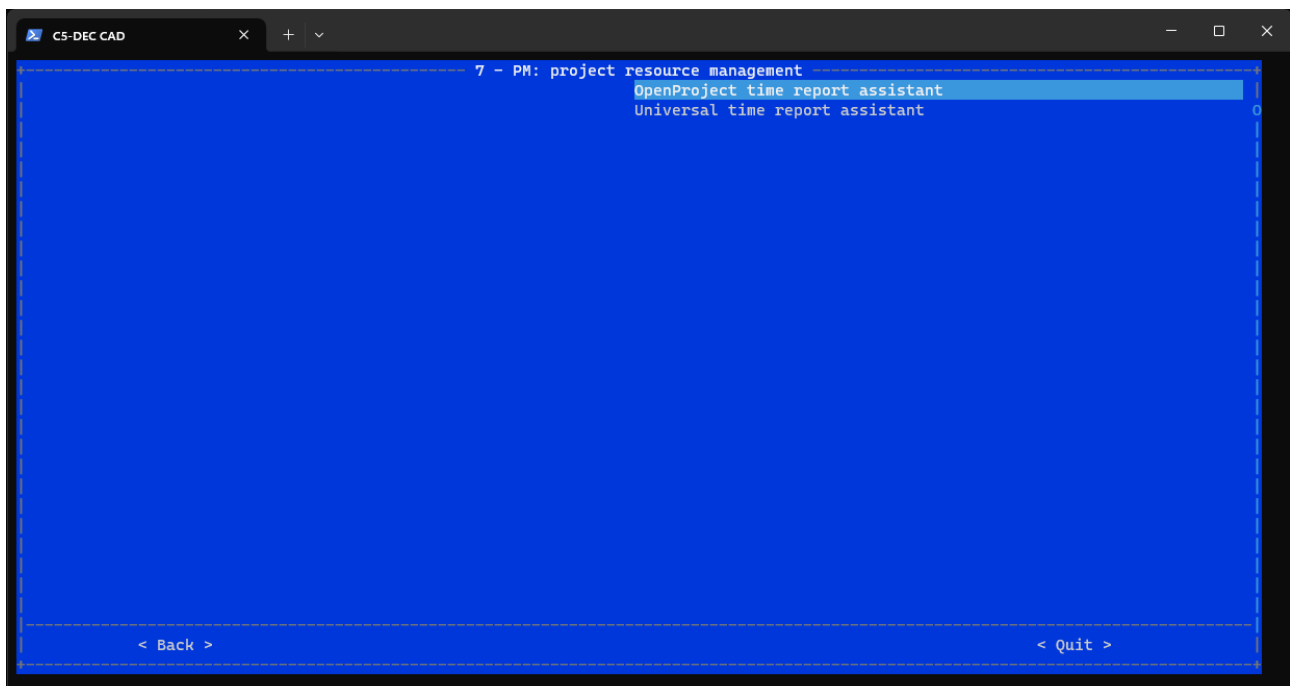


Figure 4.13: OpenProject assistant

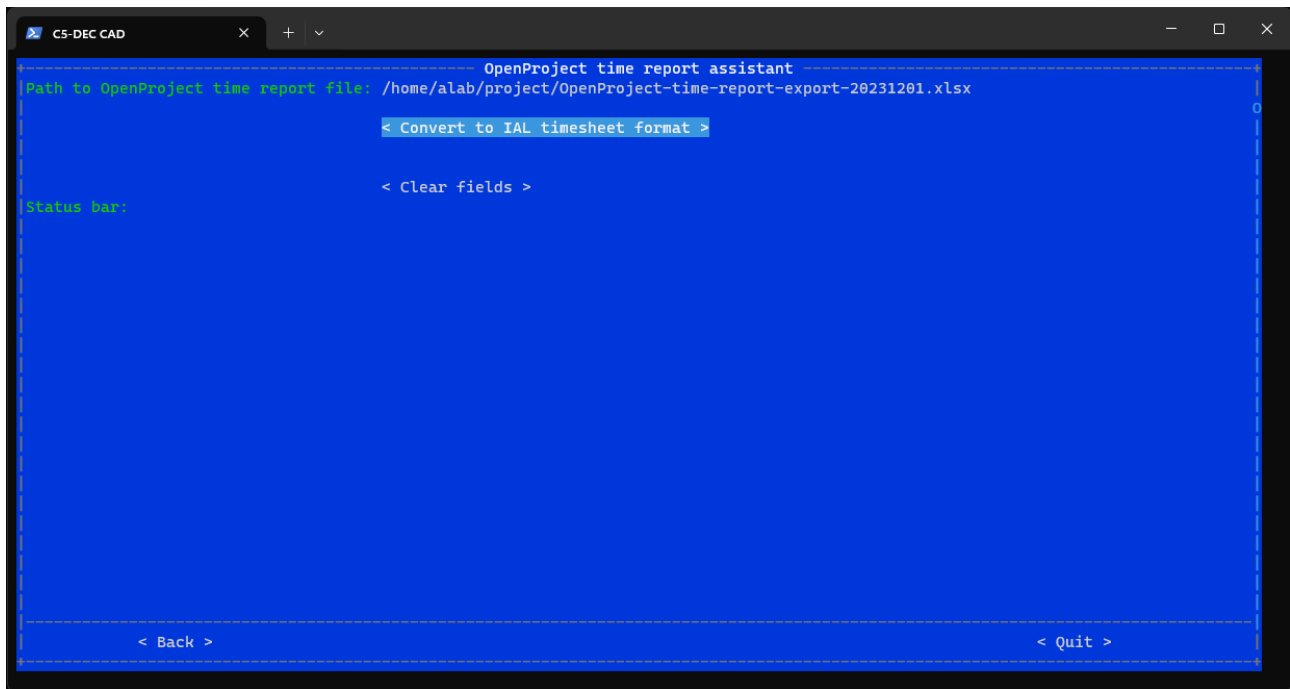


Figure 4.14: Conversion view

4.4.2 Procedure B: consolidate and filter reports

1. Open PM > Time report assistant.
2. Set path to folder containing report files.
3. Optionally enable filters and set:
 - from date,
 - to date,
 - field and field value.
4. Trigger consolidation.

CLI equivalent:

```
c5dec consolidate <name> [-l] [-f FROMDATE] [-t TO] [-d FIELD] [-v VALUE]
```

Expected outcome: - A consolidated file named consolidated-TSH-<date>-<timestamp>.xlsx is created.

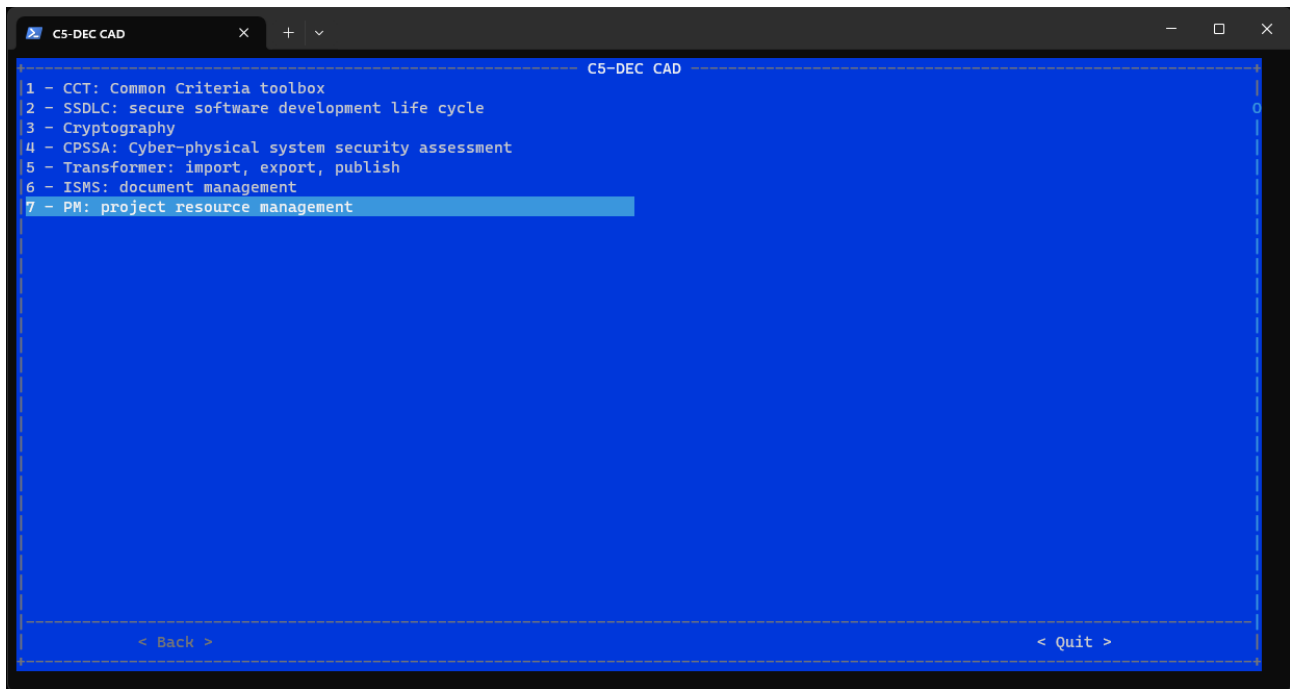


Figure 4.15: PM menu

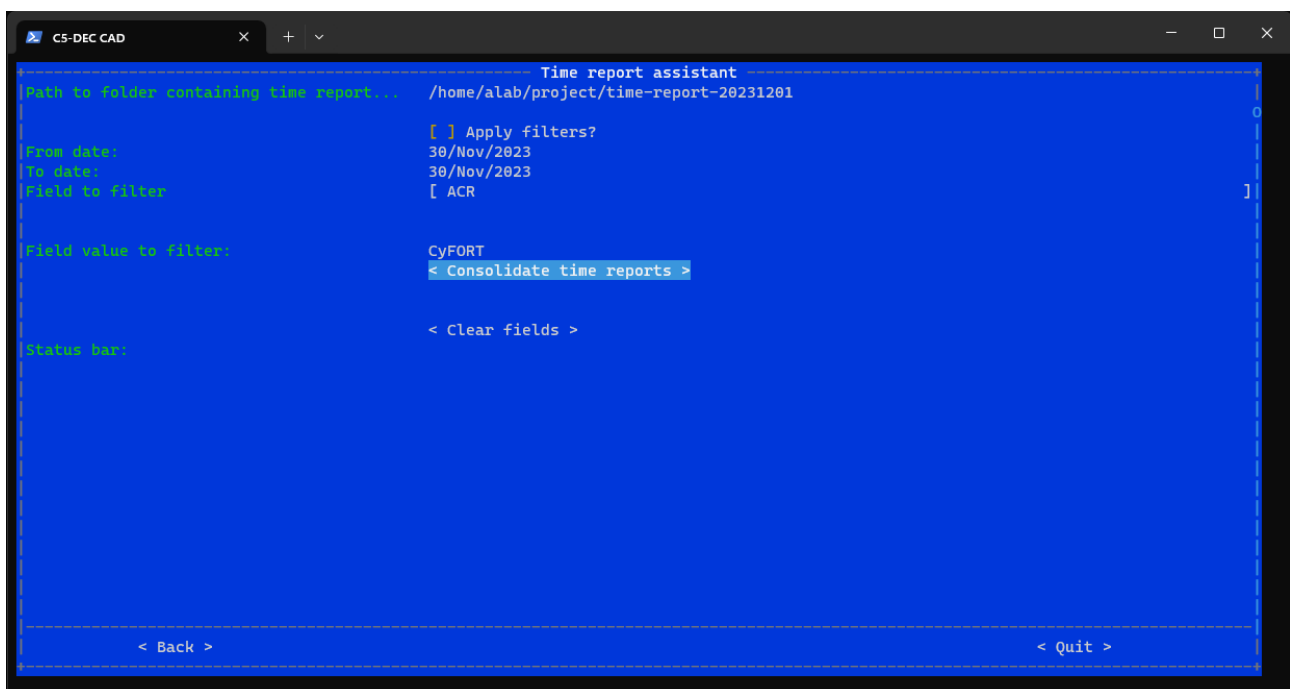


Figure 4.16: Consolidation assistant

4.4.3 Procedure C: compute cost report and analyze

1. Run cost report on consolidated timesheet:

```
c5dec costrep <name>
```

2. Review output sheets:

- CostReport

- VerificationSheet

3. Optionally copy into analysis template and refresh pivot tables for VAT view.

Expected outcome: - Cost outputs include computed daily rates, per-row costs, and verification trace columns.

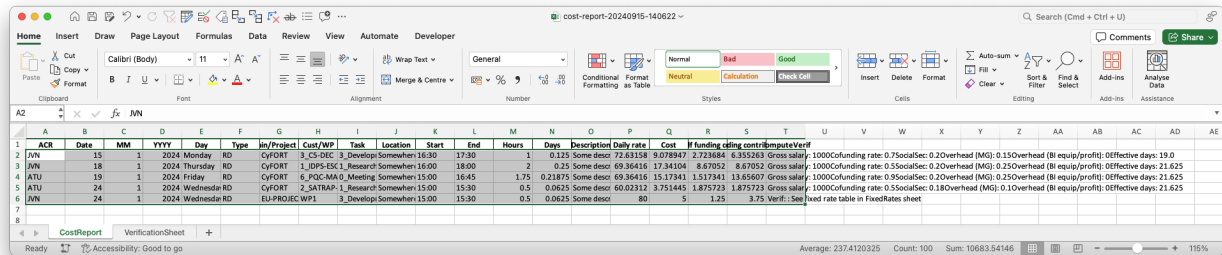


Figure 4.17: Cost report in Excel

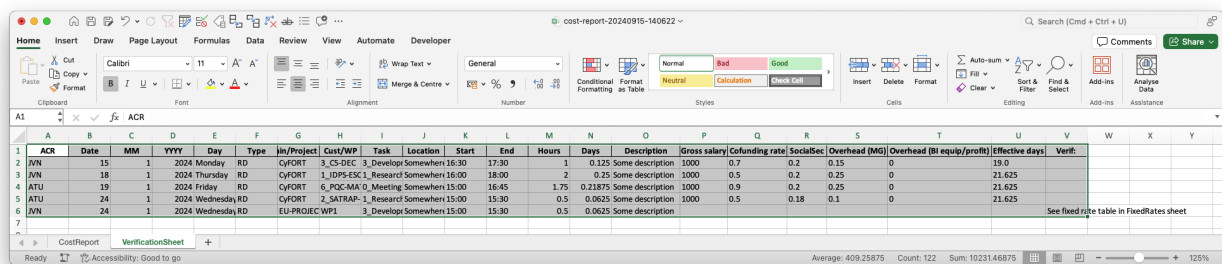


Figure 4.18: Verification sheet

4.5 Cyber-physical system security assessment (CPSSA)

Use CPSSA for threat-model generation, DFD generation, report drafting, and FAIR-based quantitative risk analysis.

Primary use:

- create threat models from architecture artifacts,
- generate reports and DFD diagrams,
- run FAIR risk analysis.

Key commands:

```
c5dec cpssa create-threat-model --project /path/to/project -o threat-model.yml
c5dec cpssa generate-report --model threat-model.yml -o cpssa-report.md
c5dec cpssa generate-dfd --project /path/to/project -o cpssa-dfd.puml
c5dec cpssa fair-input --model threat-model.yml -o fair-params.yml
c5dec cpssa risk-analysis --model threat-model.yml --fair-params fair-params.yml
```

4.5.1 Procedure A: create a threat model template

1. Generate Threatile YAML (default):

```
c5dec cpssa create-threat-model --project /path/to/project -o threat-model.yml
```

2. Optional format alternatives:

```
c5dec cpssa create-threat-model --project /path/to/project --format pytm-python  
c5dec cpssa create-threat-model --project /path/to/project --format pytm-json
```

3. If auto-discovery is not enough, pass --arc-folder explicitly.

Expected outcome: - Threat-model scaffold generated from ARC/HARC/LARC items, including assets, boundaries, and baseline scenario structure.

4.5.2 Procedure B: generate report and DFD

1. Generate markdown CPSSA report:

```
c5dec cpssa generate-report --model threat-model.yml -o cpssa-report.md
```

2. Generate PlantUML DFD:

```
c5dec cpssa generate-dfd --project /path/to/project -o cpssa-dfd.puml
```

Expected outcome: - Report and DFD artifacts aligned to architecture item semantics and zone boundaries.

4.5.3 Procedure C: FAIR calibration and quantitative risk analysis

1. Create FAIR parameter template:

```
c5dec cpssa fair-input --model threat-model.yml -o fair-params.yml
```

2. Calibrate scenario values in fair-params.yml.
3. Execute risk simulation:

```
c5dec cpssa risk-analysis --model threat-model.yml --fair-params fair-params.yml -o results/
```

Expected outcome: - Scenario-level ALE summaries, CSV simulation output, and HTML risk views.

4.6 Cryptography

Use cryptography module for integrity, signatures, encryption/decryption, secret sharing, and Ed25519 signing.

Primary use:

- SHA-256 integrity checks,
- GnuPG signing and encryption,
- Shamir secret splitting and recovery,
- NaCl Ed25519 signing and verification.

Key commands:

```
c5dec crypto hash <file>
c5dec crypto verify-hash <file> <hash>
c5dec crypto sign <file> -o <signature>
c5dec crypto encrypt <file> -r <recipient>
c5dec crypto shamir-split <secret_hex> -n 5 -k 3
c5dec crypto nacl-keygen
```

4.6.1 Procedure A: integrity and signature workflow

1. Hash and verify file integrity:

```
c5dec crypto hash path/to/file.pdf
c5dec crypto verify-hash path/to/file.pdf <expected_sha256>
```

2. Create and verify detached signature:

```
c5dec crypto sign report.pdf -o report.pdf.sig
c5dec crypto verify-sig report.pdf report.pdf.sig
```

Expected outcome: - Integrity digest and signature-verification pass/fail status.

4.6.2 Procedure B: encryption and decryption workflow

1. Encrypt for recipients:

```
c5dec crypto encrypt sensitive.docx -r alice@example.com bob@example.com
```

2. Decrypt asymmetric file:

```
c5dec crypto decrypt sensitive.docx.gpg -o sensitive.docx
```

3. Decrypt symmetric file with passphrase:


```
c5dec crypto decrypt symmetric-file.gpg -o output.txt --passphrase "my-passphrase"
```

Expected outcome: - Encrypted artifact and successful plaintext recovery.

4.6.3 Procedure C: secret sharing and NaCl signing

1. Split hex secret:

```
c5dec crypto shamir-split deadbeef -n 5 -k 3
```

2. Recover from threshold shares:

```
c5dec crypto shamir-recover "1:..." "3:..." "5:..."
```

3. Generate NaCl keypair and sign/verify message:

```
c5dec crypto nacl-keygen  
c5dec crypto nacl-sign "approved by alice" <signing_key_hex>  
c5dec crypto nacl-verify <signed_hex> <verify_key_hex>
```

Expected outcome: - Recovered secret and verified Ed25519-style signed message.

4.7 Information security management system (ISMS)

Use ISMS Python APIs to verify document inventories, build activity reports, and convert Word tags to hyperlinks.

Primary use:

- verify ISMS folder/document list consistency,
- generate activity logs,
- process Word tags into links.

4.7.1 Procedure A: verify ISMS folder against reference list

1. Run DocListAssistant in Python/Poetry shell:

```
from c5dec.core.isms import DocListAssistant  
  
assistant = DocListAssistant()  
assistant.set_path("/path/to/isms-folder")  
assistant.set_reference("/path/to/reference-doc-list.csv")  
assistant.scandir()  
assistant.save_to_csv("isms-verification-report.csv")
```

Expected outcome: - Present/missing/unlisted documents are reported and exported to CSV.

4.7.2 Procedure B: generate activity report CSV

1. Ensure activity folder follows expected dated/user hierarchy.
2. Run ActivityReport:

```
from c5dec.core.isms import ActivityReport

report = ActivityReport()
report.set_path("/path/to/activities")
report.scandir()
report.save_to_csv("activity-report.csv")
```

Expected outcome: - CSV contains date/user/event-level activity rows suitable for ISMS audit and PM reuse.

4.7.3 Procedure C: convert Word tags to hyperlinks

1. Prepare CSV mapping (tag,url,display_text).
2. Run processor:

```
from c5dec.core.isms import WordTagProcessor

processor = WordTagProcessor()
processor.set_params(
    doc_path="report-template.docx",
    csv_path="tag-mapping.csv",
    regex_text=r"\{REF: [A-Z]+-\d+\}",
    ignore_missing_tag=True,
)
processor.convert_tags_to_hyperlinks(keep_style=False)
```

Expected outcome: - .docx is updated in place with hyperlink fields.

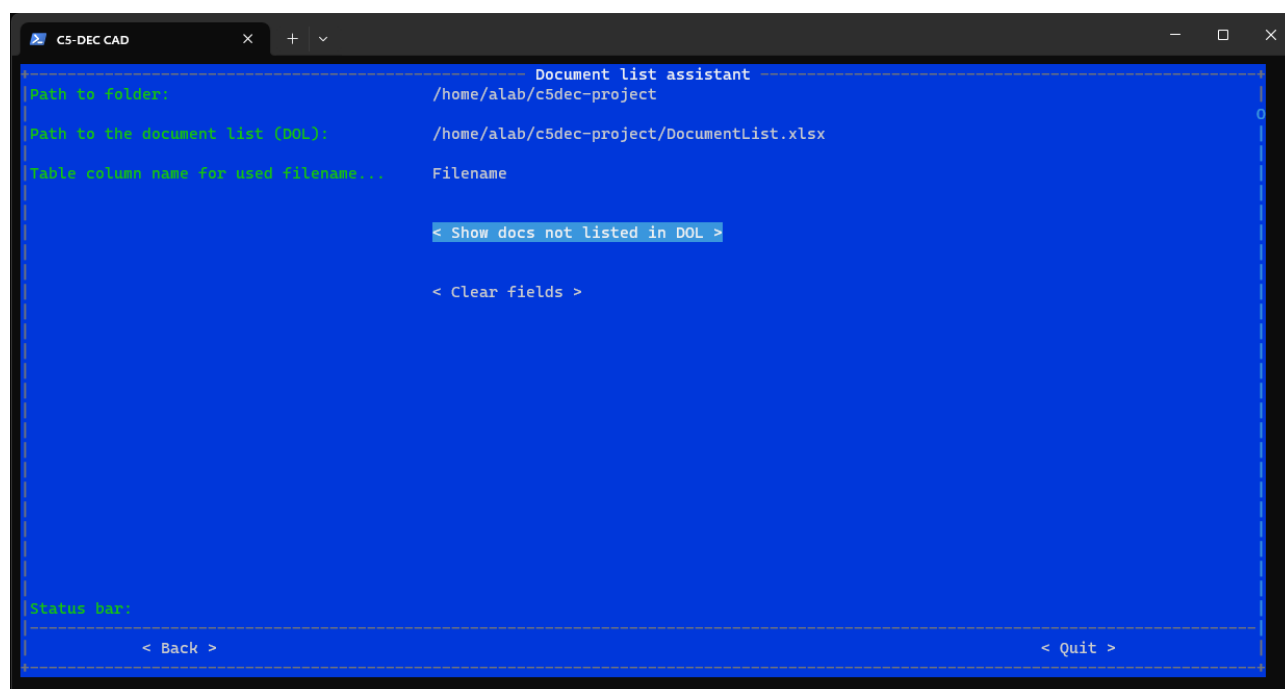
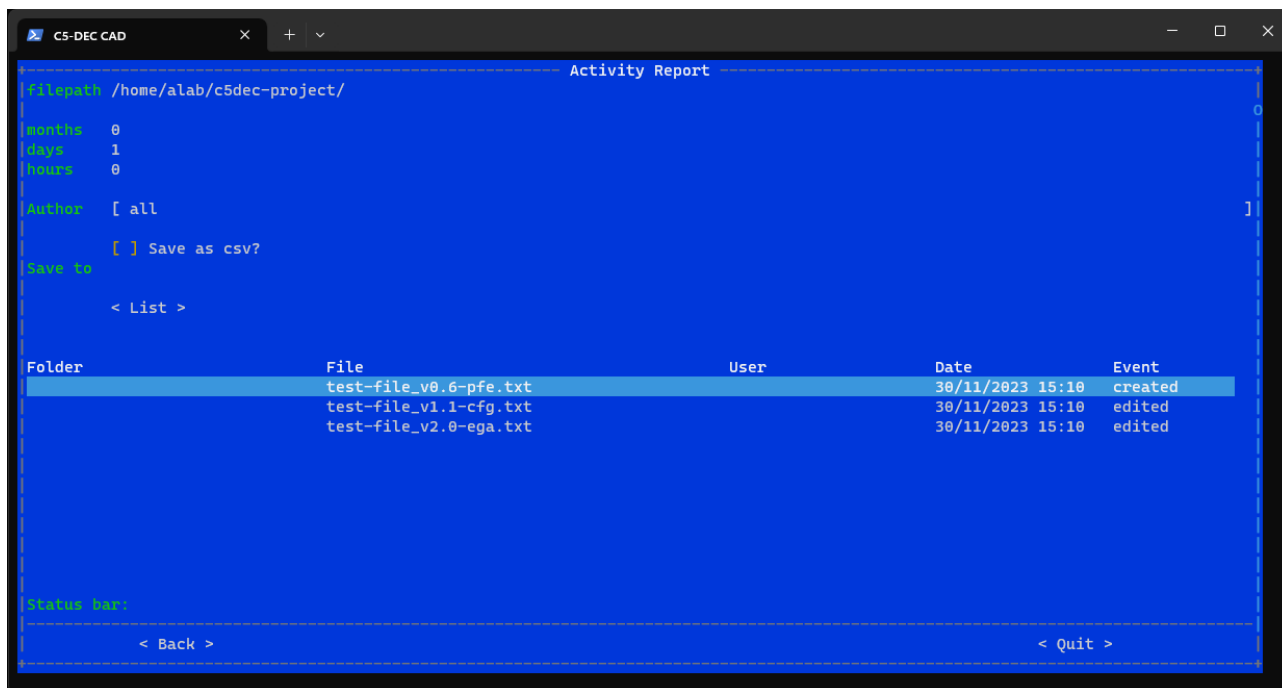


Figure 4.19: DocList assistant



Folder	File	User	Date	Event
	test-file_v0.6-pfe.txt		30/11/2023 15:10	created
	test-file_v1.1-cfg.txt		30/11/2023 15:10	edited
	test-file_v2.0-ega.txt		30/11/2023 15:10	edited

Figure 4.20: Activity report

4.8 Software bill of materials (SBOM)

Use SBOM module to generate component inventories, import to Doorstop, diff versions, and validate basic CRA transparency criteria.

Primary use:

- generate SBOMs in CycloneDX or SPDX format,
- import SBOM into Doorstop,
- diff versions and validate compliance.

Key commands:

```
c5dec sbom generate /path/to/source -o sbom_v1.0.json -f cyclonedx
c5dec sbom import sbom_v1.0.json --prefix SBOM-v1.0 --version "1.0"
c5dec sbom diff SBOM-v1.0 SBOM-v2.0 -o sbom_diff_report.md
c5dec sbom validate SBOM-v1.0
```

4.8.1 Procedure A: generate and import SBOM

1. Verify Syft availability:

```
syft version
```

2. Generate SBOM:

```
c5dec sbom generate /path/to/project -f cyclonedx -o sbom-v1.0.json
```

3. Import into Doorstop:

```
c5dec sbom import sbom-v1.0.json --prefix SBOM-v1.0 --version "1.0"
```

Expected outcome: - Doorstop SBOM document is created with one item per component.

4.8.2 Procedure B: diff and validate SBOM

1. Compare two imported SBOM versions:

```
c5dec sbom diff SBOM-v1.0 SBOM-v2.0 -o sbom-changes.md
```

2. Validate imported SBOM:

```
c5dec sbom validate SBOM-v1.0
```

Expected outcome: - Added/removed/changed component report and compliance-oriented structural checks.

4.9 CRA compliance

Use CRA module to create Annex-I checklist artifacts, integrate SBOM evidence, export compliance tables, and generate Annex-VII technical documentation.

Primary use:

- create CRA requirement checklist,
- verify CRA checklist and SBOM coverage,
- export compliance report,
- generate CRA technical documentation.

Key commands:

```
c5dec cra --create --prefix CRAC  
c5dec cra --verify  
c5dec cra --export cra_compliance_report.xlsx  
c5dec docengine cra-tech-doc -n cra-tech-doc
```

4.9.1 Procedure A: create and complete CRA checklist

1. Create checklist for product class:

```
c5dec cra --create --prefix CRAC  
c5dec cra --create --prefix CRAC --category class_i
```

2. Edit requirement item verdict/evidence fields in generated docs/specs/CRAC files.

Expected outcome: - Structured Doorstop checklist with CRA requirement mapping and review evidence.

4.9.2 Procedure B: integrate SBOM and export compliance report

1. Generate/import SBOM:

```
c5dec sbom generate /path/to/source -o sbom_v1.0.json -f cyclonedx
c5dec sbom import sbom_v1.0.json --prefix SBOM-v1.0 --version "1.0"
```

2. Run auto-verification and export:

```
c5dec cra --verify
c5dec cra --export cra_compliance_report.xlsx
```

Expected outcome: - CRA checklist includes SBOM compliance evidence and exported Excel compliance summary.

4.9.3 Procedure C: build CRA technical documentation

1. Create CRA DocEngine template:

```
c5dec docengine cra-tech-doc -n cra-tech-doc
```

2. Configure product metadata in c5dec_config.yml.
3. Review and adjust chapter content.
4. Render outputs:

```
quarto render --to pdf
quarto render --to html
quarto render --to docx
```

Expected outcome: - Annex-VII style CRA technical documentation generated in PDF/HTML/DOCX.

Traceability Statistics

Generated on 2026-03-11

1. Summary 2. MRS → ARC coverage 3. MRS → SRS coverage 4. SWD → ARC coverage 5. SRS → TCS coverage 6. TCS → TRP coverage 7. Defect severity summary 8. Overall health score

1. Summary — unique items per document type

Document type	Unique items
MRS	65
ARC	33
SRS	61
SWD	27
TCS	86
TRP	87

2. MRS → ARC coverage

51/65 (78.5%) 14 MRS item(s) are no ARC

MRS	Linked ARCs	Status
MRS-001	ARC-003, ARC-004	✓ covered
MRS-002	ARC-003, ARC-004	✓ covered
MRS-003	ARC-003, ARC-004	✓ covered
MRS-004	ARC-003	✓ covered

Figure 4.21: Traceability statistics

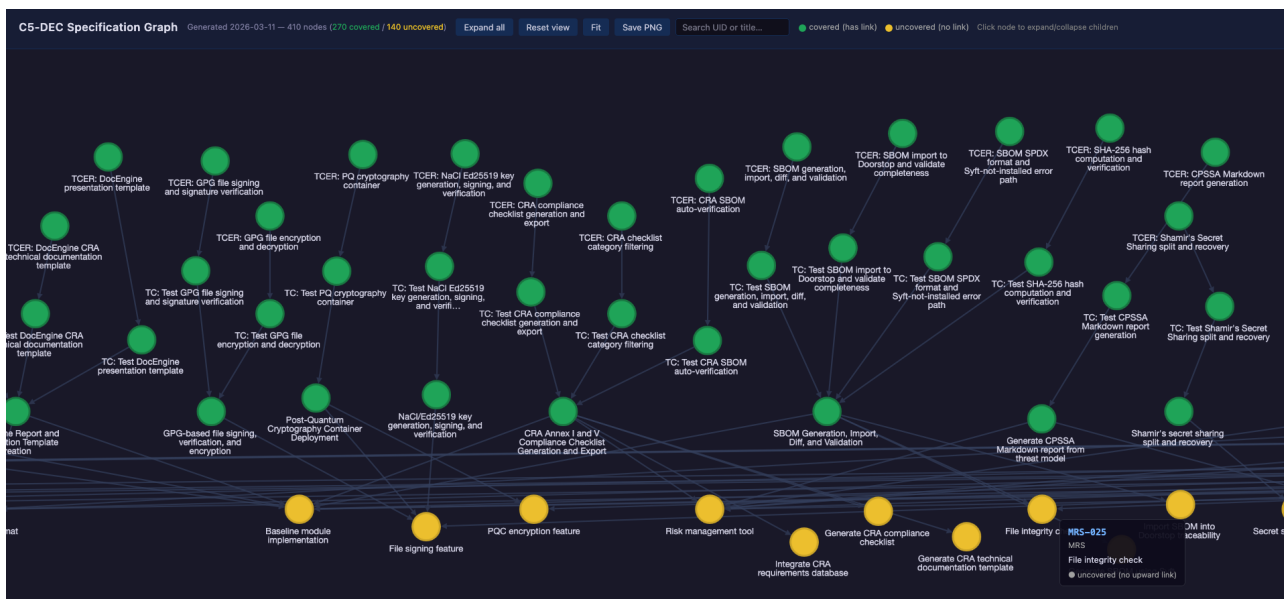


Figure 4.22: Traceability graph

Chapter 5

Troubleshooting

This chapter reorganizes the troubleshooting page into a quick triage sequence.

5.1 Fast triage order

1. Check container and Docker status.
2. Verify Poetry environment and command availability.
3. Validate Doorstop consistency when specs-related commands fail.
4. Check GUI port allocation and browser reachability.

5.2 Container build failures

Typical issues from the manual:

- Docker daemon unavailable.
- Visual Studio Code cannot reopen in container.
- Network failures during container build.
- Package installation failures in dev.Dockerfile stages.

5.3 Poetry environment issues

Typical issues from the manual:

- `c5dec: command not found` inside container.
- `poetry install` dependency conflict.
- Editor import warnings caused by interpreter mismatch.

5.4 Doorstop validation errors

Typical issues from the manual:

- unknown parent document,
- broken links,

- YAML parse failures.

When this happens, validate structure first, then inspect item links and YAML formatting before re-running publish steps.

5.5 GUI port conflicts

Typical issues from the manual:

- address already in use,
- browser cannot reach local GUI endpoint.

Use an available local port and verify the selected launch mode in your shell session.

5.6 General recovery guidance

When in doubt:

1. return to a clean shell in the project root,
2. reactivate Poetry,
3. re-run the smallest command that reproduces the issue,
4. only then escalate to full publish or render workflows.

Chapter 6

Command reference

This quick reference aggregates frequently used commands from the manual pages.

6.1 Global launcher

```
c5dec -h  
c5dec -t  
c5dec -g
```

6.2 CCT

```
c5dec view <ID>  
c5dec validate -d <IDs>  
c5dec checklist -c <prefix> --id <IDs> --info <info>  
c5dec checklist <prefix> --validate  
c5dec export <name> <version> -p <components>  
c5dec etr -f <families> -t <tables>
```

6.3 SSDLC and publishing

```
c5dec new -p <project_name> -u <user_directory>  
poetry run doorstop  
cd docs/specs && ./publish.sh
```

6.4 PM

```
c5dec timerep <openproject-export.xlsx>  
c5dec consolidate <input-folder>  
c5dec costrep <timesheet.xlsx>
```

6.5 CPSSA

```
c5dec cpssa create-threat-model --project <project>
c5dec cpssa generate-report --model <model>
c5dec cpssa generate-dfd --project <project>
c5dec cpssa fair-input --model <model>
c5dec cpssa risk-analysis --model <model> --fair-params <params>
```

6.6 Cryptography

```
c5dec crypto hash <file>
c5dec crypto verify-hash <file> <hash>
c5dec crypto sign <file> -o <signature>
c5dec crypto verify-sig <file> <signature>
c5dec crypto encrypt <file> -r <recipient>
c5dec crypto decrypt <file.gpg> -o <file>
c5dec crypto shamir-split <secret_hex> -n <n> -k <k>
c5dec crypto shamir-recover <share1> <share2> <share3>
c5dec crypto nacl-keygen
c5dec crypto nacl-sign <message> <signing_key_hex>
c5dec crypto nacl-verify <signed_hex> <verify_key_hex>
```

6.7 CRA and SBOM

```
c5dec cra --create --prefix <prefix> [--category default|class_i|class_ii|critical]
c5dec cra --verify [--prefix <prefix>]
c5dec cra --export <report.xlsx>
c5dec sbom generate <target> -o <sbom.json> -f cyclonedx|spdx
c5dec sbom import <sbom.json> --prefix <prefix> --version <version>
c5dec sbom diff <sbom1> <sbom2> -o <diff.md>
c5dec sbom validate <prefix>
c5dec docengine cra-tech-doc -n <output_name>
```

6.8 Notes

- Some commands assume a Poetry-activated environment.
- Some rendering and publishing paths assume Quarto and LaTeX dependencies are installed.
- The manual notes that `c5dec publish` is currently feature-flagged off.

Chapter 7

References

7.1 Source manual pages

- docs/manual/README.md
- docs/manual/installation.md
- docs/manual/start.md
- docs/manual/cct.md
- docs/manual/ssdlc.md
- docs/manual/pm.md
- docs/manual/cpssa.md
- docs/manual/cryptography.md
- docs/manual/isms.md
- docs/manual/sbom.md
- docs/manual/cra.md
- docs/manual/troubleshooting.md

7.2 Figure source

All screenshots in this report are sourced from docs/manual/_figures.